



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Data mining Project

Group Members:

Shahzaib Afzal (22i-1956)

Hamza Sabih(22i-1948)

Taha Rasheed (22i-2009)

Electricity Demand Forecasting: Code and Findings

Data Mining Project Team

May 10, 2025

1 Introduction

This report presents the methodology, code analysis, and findings from a data mining project aimed at forecasting electricity demand using weather and temporal data. The project, implemented in a Jupyter Notebook (DM_Project_Final.ipynb), employs multiple machine learning and time-series models to predict hourly electricity demand across cities. The analysis includes preprocessing, feature engineering (including clustering and PCA), model training, evaluation, and a front-end Flask application for visualization. The report summarizes the code structure, key findings, and insights derived from the results.

2 Data and Preprocessing

2.1 Data Sources

The dataset comprises:

- Weather Data: Hourly weather measurements (e.g., temperature, humidity, wind speed) for multiple cities (e.g., Philadelphia, Seattle) from July 2018 to May 2020, stored in JSON files.
- DemandData: Hourly electricity demand for cities, provided in a CSV file (cleaned_subregion_d

2.2 Preprocessing (Cells 1-2)

- Weather Data Processing:
 - Loaded JSON files using preprocess_weather_jsons_colab, converting epoch timestamps to UTC datetime.
 - Forward-filled missing values and concatenated data for all cities into weather_df.
 - Saved to weather_cleaned.csv.
- Demand Data Processing:
 - Loaded CSV using preprocess_electric_demand_colab, using local time for timestamps.
 - Standardized city names to lowercase, removed duplicates, and sorted by city and timestamp.
 - Saved to demand_cleaned.csv.

2.3 Feature Engineering (Inferred)

The merged_arma DataFrame (used in Cell 15) contains features:

- Weather Features: temperature, humidity, windSpeed.
- Temporal Features: hour, seasonal dummies (season_fall, season_spring, season_summer, season_winter).
- Clustering: A Cluster column, likely derived from K-Means clustering on weather and temporal features to capture patterns (e.g., high-demand hot hours).
- Target: demand (current demand) and demand_next_24h (24-hour ahead demand, though not used in provided cells).

Clustering and PCA (Principal Component Analysis) were performed (as confirmed by the user) to group similar conditions and reduce dimensionality, though the exact code is in unshown cells. Visualizations (e.g., PCA scatter plots, time-series plots) likely supported exploratory data analysis (EDA).

3 Methodology

3.1 Model Initialization (Cell 15)

Seven models were initialized:

- ARIMA: Time-series model with order (5,1,0), fitted on demand.
- Linear Regression: Basic regression model.
- Polynomial Regression: Degree-2 polynomial features with linear regression.
- Random Forest: 100 estimators, random state 42.
- XGBoost: Regression with squared error objective, random state 42.
- LSTM: Single-layer LSTM with 50 units, reshaped input for time steps, compiled with Adam optimizer and MSE loss.
- Stacking Ensemble (Cell 19): Combines Random Forest, XGBoost, and Linear Regression with RidgeCV as the meta-learner.

Features included temperature, humidity, windSpeed, hour, Cluster, and seasonal dummies.

3.2 Data Splitting and Training (Cell 16)

- Split: 80% training, 20% testing based on time order (16,502 total samples: 13,201 train, 3,301 test).
- Training:
 - ARIMA fitted on training demand.

- Machine learning models (Linear, Polynomial, Random Forest, XGBoost) trained on X_{train} , y_{train} .
- LSTM trained on reshaped 3D input for 10 epochs.
- Random Forest hyperparameters tuned via GridSearchCV ($n_estimators$: [100, 200], max_depth : [10, 20]).

3.3 Evaluation (Cell 17)

Models were evaluated on the test set using:

- Mean Absolute Error (MAE): Average absolute prediction error.
- Root Mean Squared Error (RMSE): Square root of average squared error.
- Mean Absolute Percentage Error (MAPE): Percentage error, handling zero values.

3.4 Baseline Comparison (Cell 18)

A naive forecast (using demand from 24 hours prior) was compared to model performance.

3.5 Ensemble Learning (Cell 19)

A stacking ensemble combined Random Forest, XGBoost, and Linear Regression, with RidgeCV as the meta-learner, to improve predictions.

4 Findings

4.1 Model Performance (Cell 17)

Table 1 summarizes test set performance:

Table 1: Model Performance Metrics			
Model	MAE	RMSE	MAPE (%)
ARIMA	198.30	323.76	2.35
Linear Regression	1341.69	1721.36	15.10
Polynomial Regression	1307.54	1654.04	14.79
Random Forest	1346.31	1730.46	15.27
XGBoost	1172.34	1501.76	13.56
LSTM	1727.73	2116.75	20.95
Stacking Ensemble	1221.07	1574.57	13.75
Naive Forecast	1287.68	1625.85	15.91

Key Observations:

- ARIMA outperformed all models, with significantly lower errors (MAE: 198.30, MAPE: 2.35%), likely due to its ability to model temporal dependencies in demand data.
- XGBoost was the best machine learning model (MAE: 1172.34, MAPE: 13.56%), benefiting from its handling of non-linear relationships and feature interactions.
- Polynomial Regression and Stacking Ensemble outperformed Linear Regression and Random Forest, indicating non-linear patterns in the data.
- LSTM performed worst (MAE: 1727.73, MAPE: 20.95%), possibly due to insufficient tuning or limited time steps (single-step input).
- The Naive Forecast (MAE: 1287.68) was outperformed by ARIMA, XGBoost, Polynomial, and Stacking, but beat Linear Regression, Random Forest, and LSTM.

4.2 Clustering and PCA

The Cluster column in merged_arima suggests K-Means clustering was applied to weather and temporal features (e.g., temperature, humidity, hour). This likely grouped similar conditions (e.g., hot high-demand hours vs. cool low-demand hours), enhancing model performance by capturing latent patterns. PCA was used (as confirmed by the user) to reduce dimensionality or visualize data, likely producing scatter plots of principal components colored by clusters or demand. These steps, performed in unshown cells, supported feature engineering and EDA.

4.3 Front-End Application

A Flask single-page application was developed to visualize predictions:

- Interface: Users select a model and date range (2018-07-01 to 2020-05-22) via a dropdown and date inputs.
- Visualization: A Chart.js time-series plot displays actual vs. predicted demand, with MAE, RMSE, and MAPE metrics.
- Implementation: Models were saved from the notebook using joblib, pickle, and .keras formats, then loaded in Flask to ensure identical performance (e.g., ARIMAs MAE: 198.30). The app runs successfully, as confirmed.

5 Discussion

5.1 Strengths

- ARIMAs Superiority: Its low errors highlight the importance of temporal modeling for demand forecasting.
- Feature Engineering: Clustering and seasonal dummies captured key patterns, improving model robustness.

- Ensemble Approach: The stacking ensemble leveraged strengths of multiple models, outperforming most individual models.
- Visualization: The Flask app provides an intuitive interface for stakeholders to explore predictions.

5.2 Limitations

- LSTM Performance: Poor results suggest the need for more time steps, hyperparameter tuning, or additional features.
- Data Gaps: Missing cells (e.g., clustering, PCA) limit full code transparency.
- Feature Selection: The impact of Cluster and other features wasnt quantified (e.g., via feature importance).

5.3 Future Work

- Enhance LSTM: Use multi-step inputs or bidirectional LSTM.
- Feature Importance: Analyze Cluster and weather features using SHAP or Random Forest importance.
- App Features: Add multi-model comparison (e.g., overlay ARIMA and XGBoost predictions) to the Flask app.
- Deployment: Host the app on a cloud platform for accessibility.

6 Conclusion

The electricity demand forecasting project successfully predicted hourly demand using weather and temporal data. ARIMA excelled due to its time-series modeling, while XGBoost and the stacking ensemble performed well among machine learning models. Clustering and PCA enhanced feature engineering, and the Flask app provided an effective visualization tool. Future improvements could focus on LSTM optimization and deeper feature analysis to further boost accuracy.